T-Lab 2026. RecSys. Исследование по автоматизации поиска фичей для градиентного бустинга и/или поиск альтернативы бустингу в задаче рекомендательной системы.

1. Вступление

Изначально с помощью ChatGPT и поисковика был проведен поиск готовых систем, которые идеологически были похожи на представленные направления на странице с заданием. Идея с трансформером (3я идея) мне не понравилась, поэтому я решил проверять первые две.

Найдя RelBench (репозиторий проекта) и LLM-FE (репозиторий проекта), я решил для сравнения двух методов между собой и бейзлайном, построенном на ручных фичах и LightGBM, взять датасет и одинаковую задачу именно у RelBench, потому что мне показалось более просто адаптировать LLM-FE под эти вводные, чем наоборот, и потому что в статье о RelBench был эксперт, который вручную подбирал под некоторые задачи фичи. А потом модель GNN как раз сравнивалась с LightGBM таких фичах.

Правда для задачи rel-hm / user-item-purchase (ранжирование товаров для покупателя по покупкам следующей недели), наиболее подходящей к нашей задаче рекомендательной системы, ручных фичей, построенных специалистом, не оказалось, поэтому пришлось чуть чуть адаптировать фичи из других задач к этой задаче. Несмотря на адаптацию, мне кажется это более справедливым бэйзлайном, чем фичи от gpt, ибо gpt - не человек, а инструмент, поэтому мне показалось странным брать результаты работы gpt за бейзлайн, если мы хотим сравнивать именно человечекие результаты с результатами, сделанными с помощью инструментов.

Основная метрика — MAP@12, среднее AP@12 по наблюдениям «покупатель — неделя». Все значения MAP в таблицах ниже приведены **в процентах**; разницы — в процентных пунктах.

Исследовательская линия была следующей:

воспроизведение RelBench

ручные признаки и GNN-эмбеддинги для LightGBM

двухбашенная модель

генерация признаков локальными и API LLM

ContextGNN, Recformer и TDM на малой выборке

сравнение на другой, большей временной выборке

диагностика повторных/новых товаров и изменения ContextGNN

Эксперименты проводились на разных рахмерах выборок: для small выборки использовались validation 7 сентября и test 14 сентября 2020 года, восемь train-срезов. В medium выборке когорта и каталог фиксировались по истории к 29 июня 2020 года; outer-test: **17 августа, 31 августа и 14 сентября 2020 года**, inner validation — предыдущая неделя, обучение — шесть предшествующих недельных срезов. Это даты данных.

Таблицы данных и названия столбцов

Ниже названия столбцов переведены на русский язык для описания данных. В самих файлах сохранены исходные имена. Состав столбцов трёх исходных таблиц одинаков для полного набора, small и medium.

Покупатели — 7 столбцов:

id покупателя;

дополнительный флаг покупателя;

флаг активности;

статус участия в клубной программе;

частота получения модных новостей;

возраст;

почтовый индекс.

Товары — 25 столбцов:

id товара;

код продукта;

название продукта;

код типа товара;

название типа товара;

группа товаров;

код графического оформления;

название графического оформления;

код цветовой группы;

название цветовой группы;

код воспринимаемой характеристики цвета;

название воспринимаемой характеристики цвета;

код основного воспринимаемого цвета;

название основного воспринимаемого цвета;

код отдела;

название отдела;

код ассортиментного раздела;

название ассортиментного раздела;

код группы ассортиментных разделов;

название группы ассортиментных разделов;

код секции;

название секции;

код группы одежды;

название группы одежды;

подробное описание товара.

Покупки — 5 столбцов:

дата покупки;

id покупателя;

id товара;

цена покупки;

код канала продаж.

Целевые таблицы обучения, валидации и тестирования — по 3 столбца:

дата прогноза;

id покупателя;

список id товаров, купленных в следующие семь дней.

Каждая строка целевой таблицы соответствует покупателю на определённую дату. Список будущих покупок является целевой переменной, а не входным признаком.

Подготовленные обучающие пары для деревьев — 4 столбца:

id покупателя;

id товара;

дата прогноза;

метка покупки.

Здесь id товара — одно значение, а не список. Метка равна 1 для положительной пары и 0 для сэмплированной отрицательной пары. Вычисленные ручные и LLM-признаки формировались отдельно и различались между моделями.

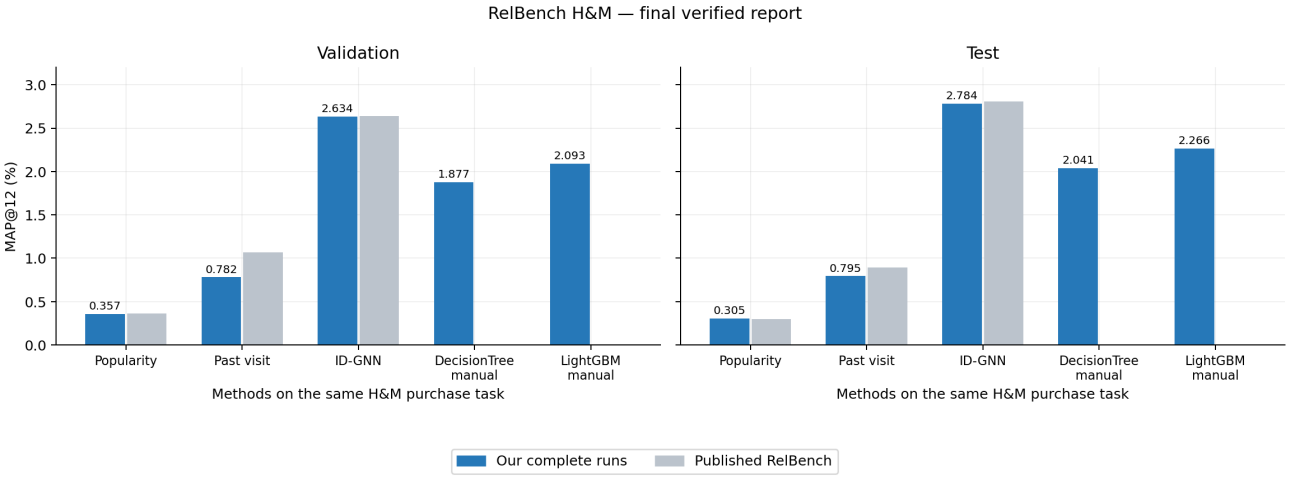
2. Полный RelBench: воспроизведение и ручные baseline

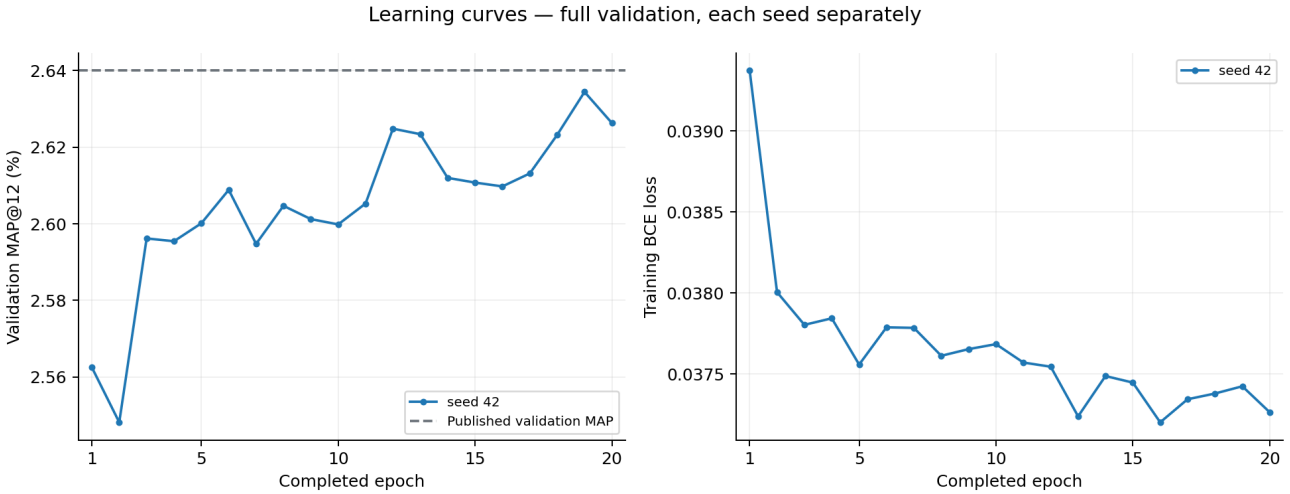
RelBench — набор задач для машинного обучения на связанных таблицах. Основная идея исследуемого подхода — представить строки таблиц как узлы графа, связи по ключам — как рёбра и обучить GNN извлекать признаки из этих связей вместо их ручного составления.

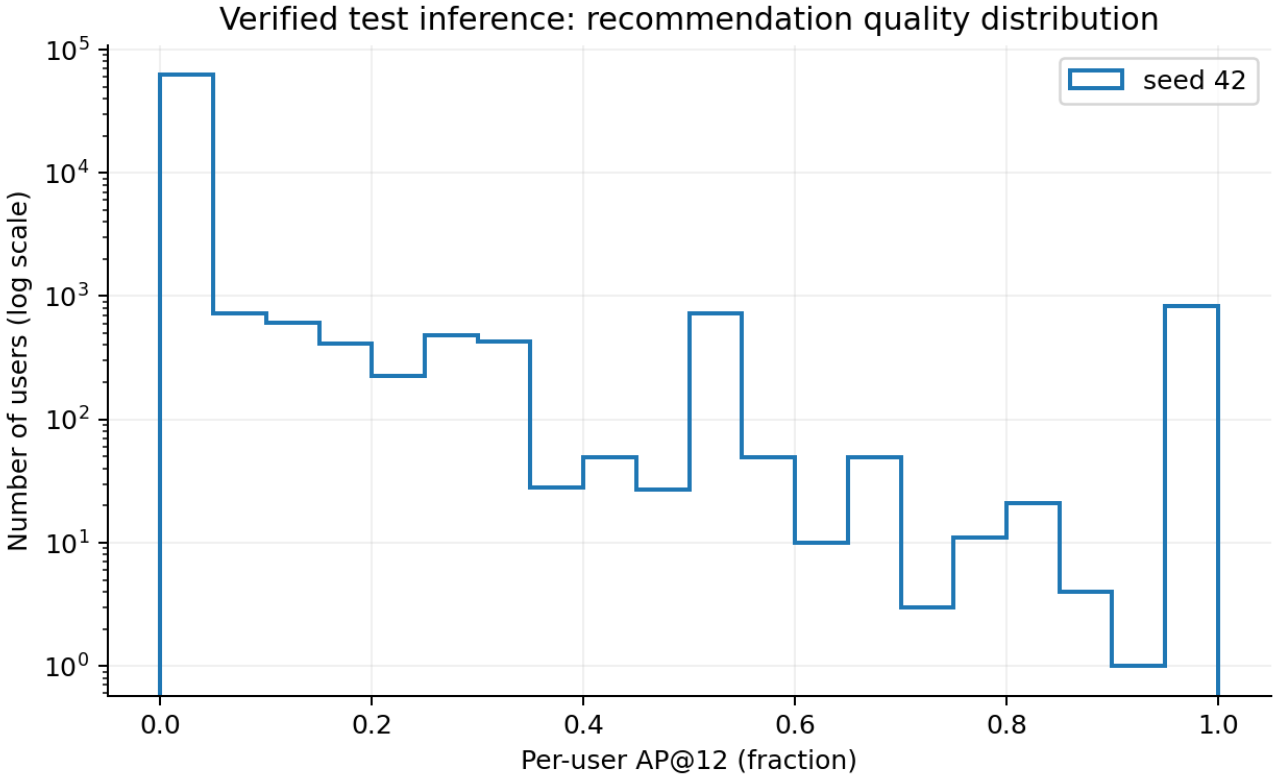
Изначально я просто хотел воспроизвести результат исследователей работы RelBench, чтобы убедиться, что всё сделано правильно и результаты примерно такие же, как в статье.

Метод	Validation MAP@12, %	Test MAP@12, %
Global popularity	0,35739	0,30456
Past visit	0,78166	0,79466
ID-GNN, seed42	2,63442	2,78432
DecisionTree, 126 ручных признаков	1,87661	2,04076
LightGBM, 126 ручных признаков	2,09318	2,26608

В проекте для статьи RelBench записаны ID-GNN validation/test **2,64% / 2,81%**, среднее пяти запусков. Локальный test ниже на 0,02568 п.п. Это близкое численное совпадение одного запуска. Дополнительно локально использовался temporal sampler last (Берёт самых последних по времени среди допустимых), тогда как в статье указан uniform (Случайно выбирает среди всех допустимых с одинаковой вероятностью), и evaluation batch 128 вместо 512. Тем не менее результат меня удовлетворил.


RelBench H&M — final verified report
Validation
Test
MAP@12 (%)
0.357
0.782
2.634
1.877
2.093
0.305
0.795
2.784
2.041
2.266
Popularity
Past visit
ID-GNN
DecisionTree manual
LightGBM manual
Methods on the same H&M purchase task
Our complete runs
Published RelBench


Learning curves — full validation, each seed separately
Validation MAP@12 (%)
Completed epoch
seed 42
Published validation MAP
Training BCE loss


Verified test inference: recommendation quality distribution
seed 42
Number of users (log scale)
Per-user AP@12 (fraction)

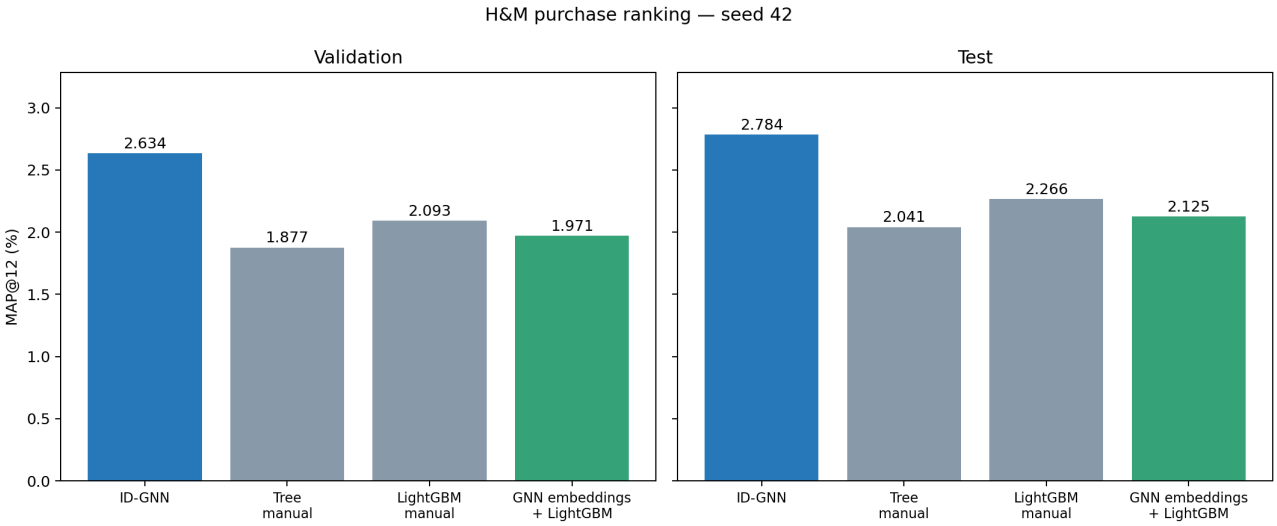
3. Полный набор: GNN → LightGBM и Two-tower

3.1. Замороженные представления ID-GNN → LightGBM

Экспортировали 128 значений перед линейной головой ID-GNN и флаг доступности представления (1 — пара присутствовала в выбранном подграфе, поэтому для неё получен вектор из 128 чисел, 0 — представления пары не было; вместо него записывался вектор из 128 нулей), всего 129 входов. Логит головы в дерево не подавался. Использованы прежние 803 377 train-пар и общие evaluation-кандидаты.

Метод	Validation, %	Test, %
ID-GNN	2,63442	2,78432
LightGBM manual	2,09318	2,26608
ID-GNN embeddings + LightGBM	1,97116	2,12451

Связка не улучшила результат: относительно manual test −0,14157 п.п., относительно GNN −0,65981 п.п. У 565 051 из 803 377 train-пар представление отсутствовало и заменялось нулями; в validation/test отсутствовало примерно по 10,7%.



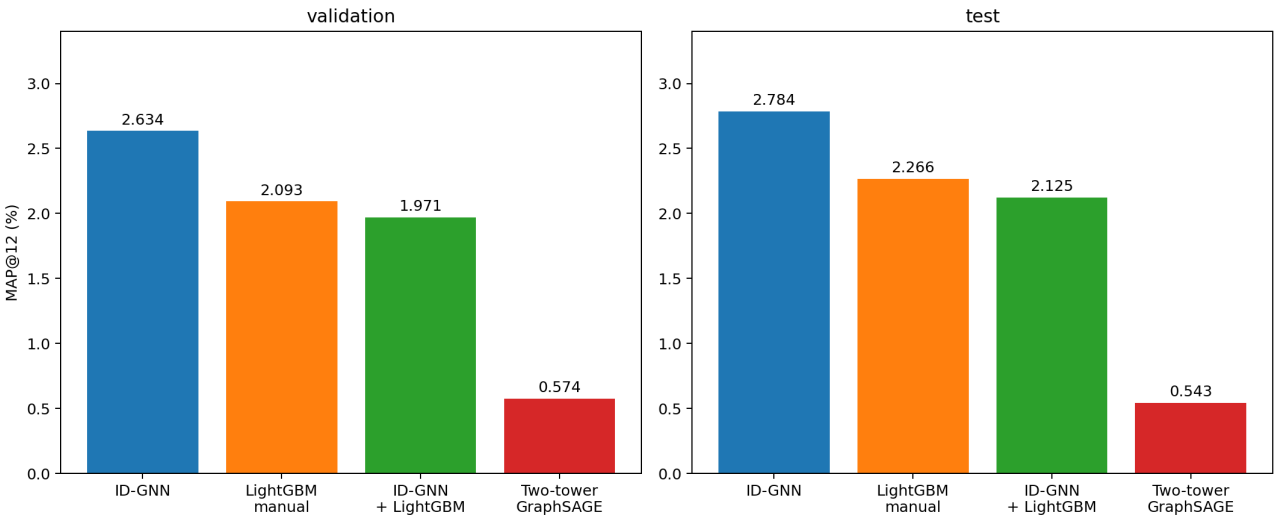
3.2. Двухбашенная GraphSAGE с общими весами

В двухбашенной архитектуре одна ветвь кодирует покупателя, другая — товар. Их векторы сравниваются скалярным произведением: чем выше результат, тем выше товар в рекомендациях. В этом варианте обе ветви используют общие веса GraphSAGE, но обрабатывают покупателя и товар отдельно.

Исследовательская основа этого варианта — RelBench, раздел 3: двухбашенная GNN оценивает пары скалярным произведением представлений и обучается с BPR loss. Наш запуск адаптирован из официального примера gnn_recommendation.py. Сам графовый encoder GraphSAGE предложен в работе Hamilton, Ying и Leskovec «Inductive Representation Learning on Large Graphs»: представление узла строится агрегацией признаков выбранных соседей. Эта исходная статья описывает GraphSAGE в целом; конкретная рекомендательная двухбашенная конфигурация здесь взята из RelBench.

Отдельные forward для пользователя и товара с общим GraphSAGE, dot product 128-мерных векторов, BPR loss и shallow item embedding. Полные 20 эпох, batch 256, ровно 2 000 шагов на эпоху, uniform sampling.

Validation/test: **0,57431% / 0,54301%**. Модель ранжировала все 105 542 товара; у прежних ID-GNN/manual использовался локальный pool. Поэтому цифры не являются чистой абляцией головы при одинаковых кандидатах.



4. LLM-FE: первые эксперименты с локальной LLM на полном наборе

Идея LLM-FE — поручить языковой модели придумывать признаки и писать код их вычисления. Каждый корректный набор проверяется обучением LightGBM и оценкой на валидации; результат помогает LLM предложить следующий вариант. Так ручной подбор признаков заменяется итеративным поиском с обратной связью. В нашем проекте эта идея адаптирована к связанным таблицам и истории покупок H&M.

Qwen2.5-Coder-7B Q4_K_M работала через отдельный Ollama. LLM формировала SQL-агрегаты истории и производные формулы, LightGBM обучалась на прежних 803 377 парах. Саму LLM не дообучали.

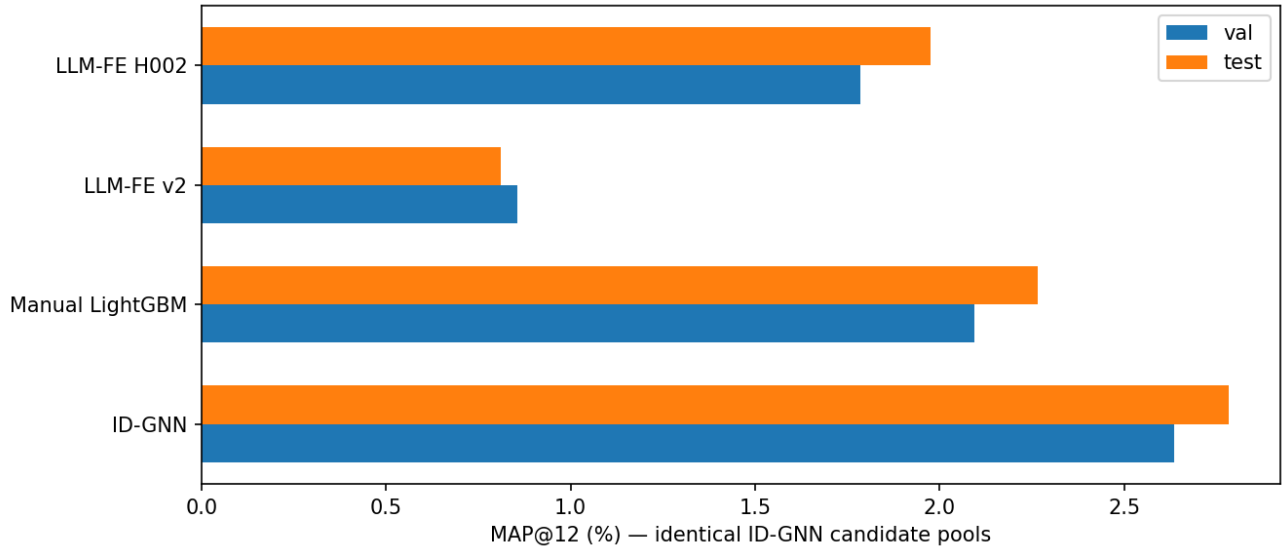
Из 20 предложений измерены только пять, остальные 15 отклонены до MAP. Первые причины: OVER — 6, история вне namespace derived — 2, группировки — 2, структура программы — 2, подзапрос — 1, строковый результат — 1, ошибка валидатора WITHIN GROUP — 1. Исправление последней само по себе не делало весь ответ корректным.

Выбран лучший кандидат. Вместо полной каталожной оценки была выполнена выполнена отдельная полная оценка на общих кандидатах ID-GNN:

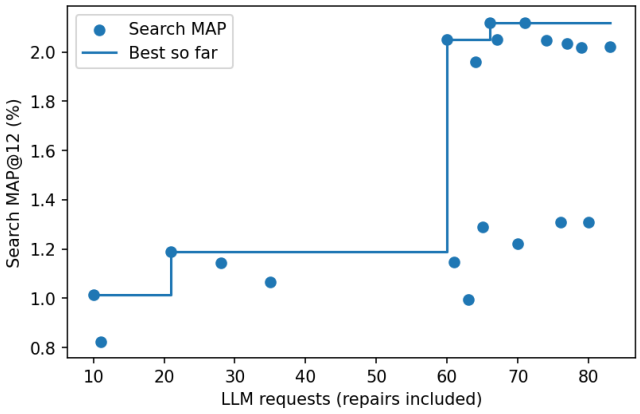
Метод на общих кандидатах	Validation, %	Test, %
ID-GNN	2,63442	2,78432
Manual LightGBM	2,09318	2,26608
LLM-FE v2	0,85428	0,81175
LLM-FE H002	1,78605	1,97540

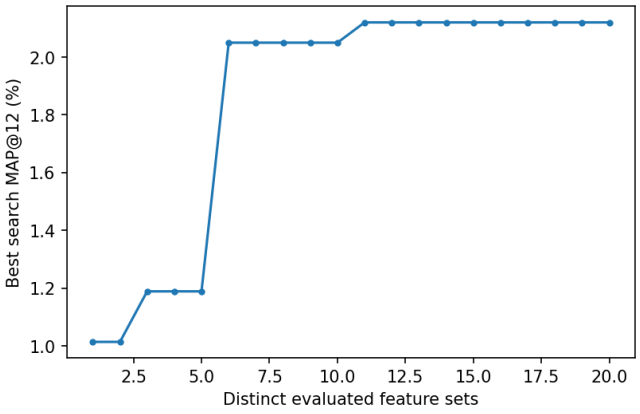
H002: ввели бюджет именно успешно оценённых различных наборов, repair loop, уточнили prompt/валидатор и уменьшили поисковый pool до кандидатов ID-GNN. Получены 20 наборов за 83 запроса, 45 repair calls; выбран candidate060. Поиск — 256 validation-пользователей, confirmation — другие 1 024. Полные val/test и точный независимый replay прошли.

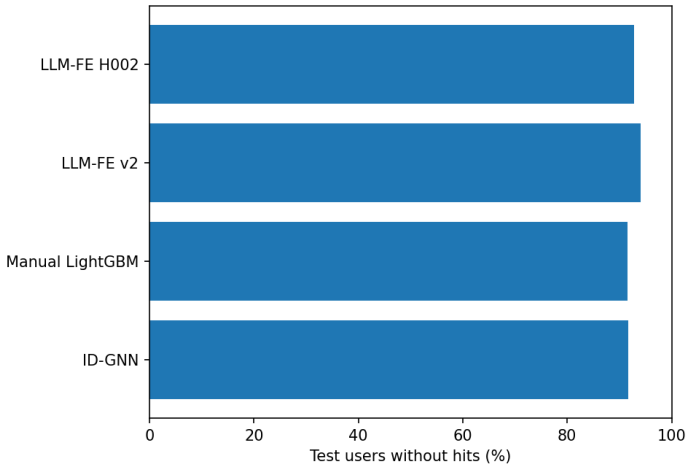
Относительно v2 test вырос на **1,16365 п.п.**, но остался ниже manual на 0,29068 п.п. Эффект нельзя приписать только repair loop: одновременно изменились бюджет, интерфейс и кандидаты поиска.

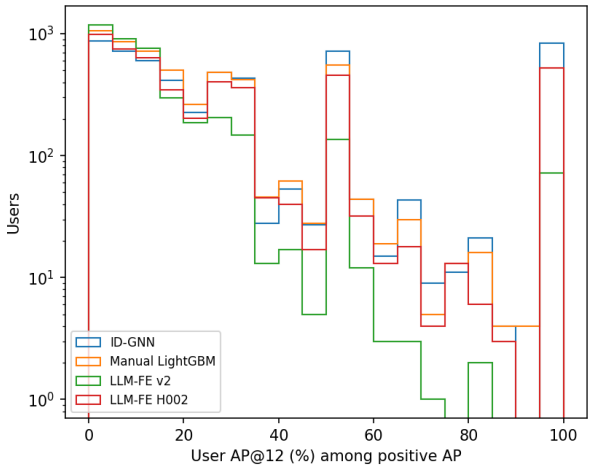


H002: fixed 256 validation users, shared ID-GNN pools; not test









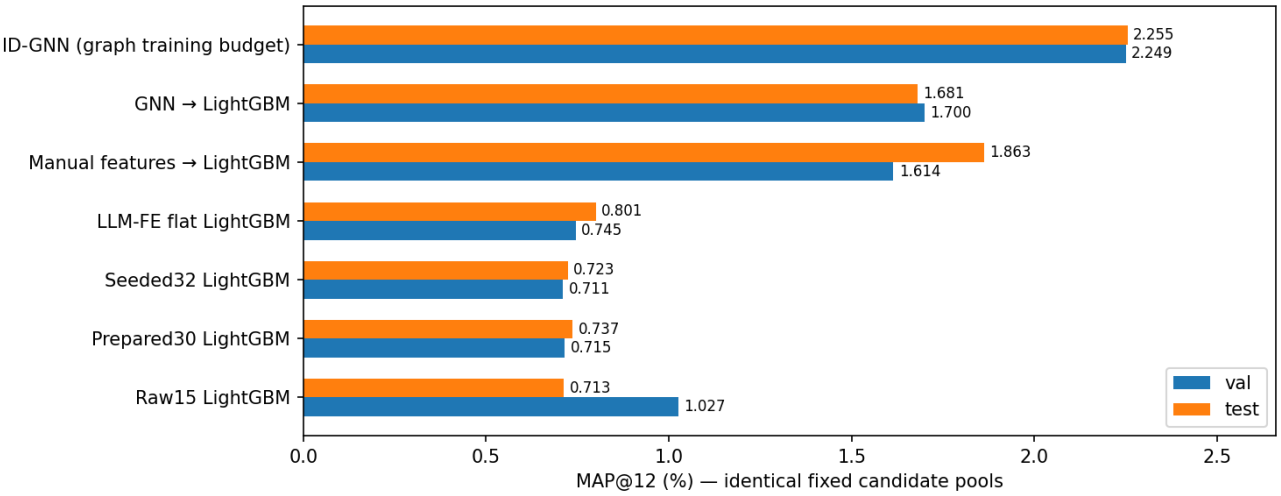
5. Kaggle Llama: отдельная подвыборка полного набора

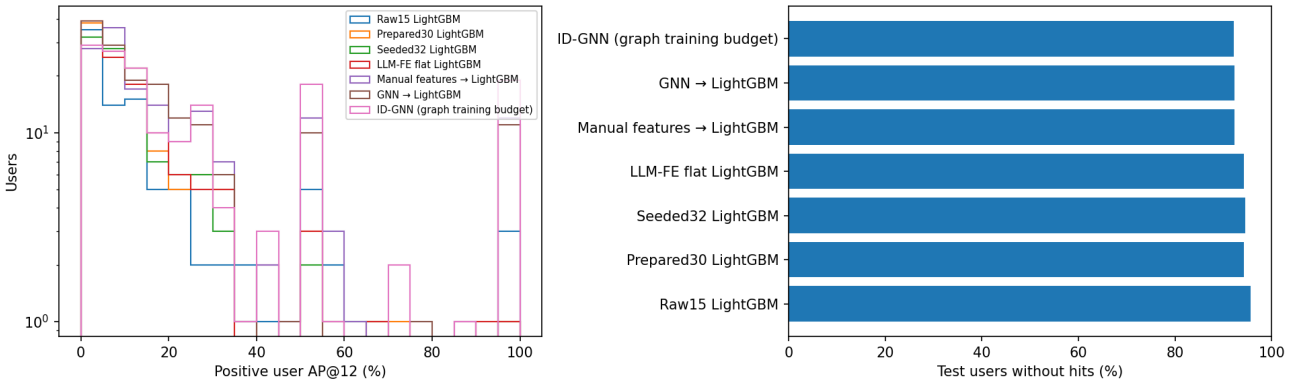
Здесь итоговые метрики относятся к **val 1 024 / test 2 048 и общим кандидатам ID-GNN**, а не к полному test или последующему small-v1. Train-пары прежние — 803 377.

Были различные проблемы с этой моделью. Она октазывалась писать хорошо, запуски шли очень долго.

Метод на этой подвыборке	Описание метода	Validation, %	Test, %
Raw15	LightGBM на 15 исходных характеристиках покупателя и товара, без исторических агрегатов.	1,02687	0,71314
Prepared30	LightGBM на 15 исходных характеристиках и 15 вручную заданных исторических признаках: частоты, давность, цены и предпочтения.	0,71542	0,73737
Seeded32	Prepared30 плюс два ручных логарифмических преобразования счётчиков покупок пользователя и товара; всего 32 признака.	0,71077	0,72335
Llama flat	Llama преобразует готовую плоскую таблицу, затем LightGBM ранжирует товары. У выбранного решения 36 признаков: 30 подготовленных, два ручных логарифма и четыре отношения от Llama.	0,74474	0,80095
Manual126	LightGBM на 126 вручную заданных признаках: адаптация экспертных SQL плюс признаки пары покупатель–товар.	1,61426	1,86288
GNN → LightGBM	LightGBM на 128 значениях представления замороженной ID-GNN и флаге доступности представления; всего 129 признаков.	1,69983	1,68057
ID-GNN	Графовая нейросеть, которая учитывает конкретного покупателя при вычислении представлений его подграфа и сама оценивает товары.	2,24926	2,25494
Qwen H002, прежние прогнозы на тех же строках	LightGBM на исходных полях и признаках, предложенных Qwen в SQL-поиске с исправлениями и бюджетом 20 оценённых наборов. Ранее выбранная модель оценена на тех же пользователях без нового обучения.	1,66953	1,42981

Это не чистое сравнение силы Llama и Qwen: у Qwen другой SQL-контекст и поиск.



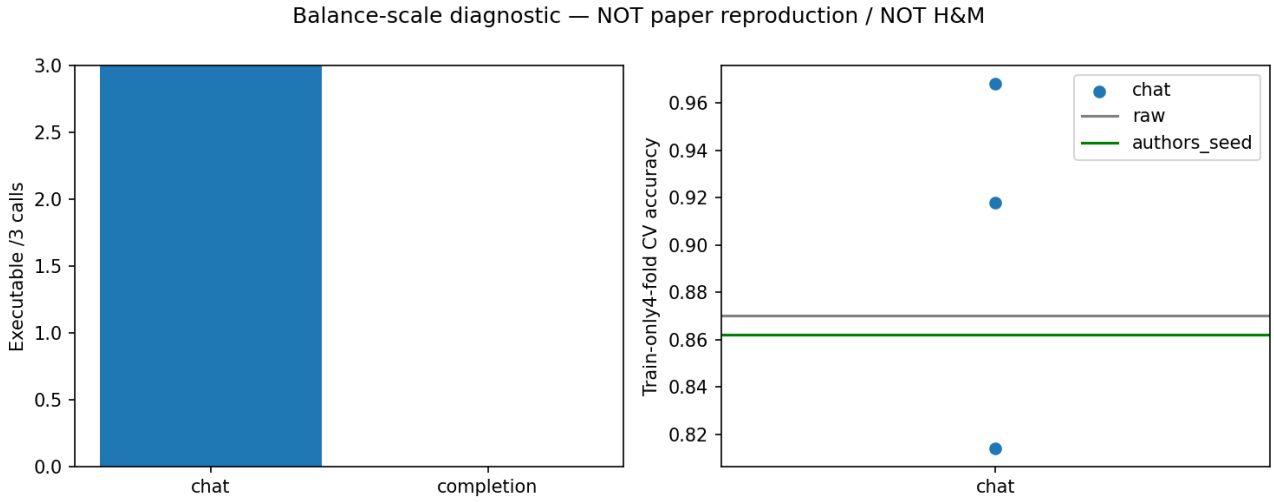


6. Balance-scale: отдельная диагностика Llama

После многочисленных ошибок Llama на H&M я решил проверить, способна ли та же модель создавать исполнимые и полезные признаки на простой задаче Balance-scale из статьи LLM-FE. Цель — проверить работу модели с понятным Python-интерфейсом и примером преобразования, чтобы сузить поиск причин неудач на связанных таблицах H&M. Это отдельная диагностика, а не полное повторение эксперимента статьи: проверены три chat-генерации, качество оценено кросс-валидацией на обучающей части, отложенный test не использовался.

Вариант	Train-only 4-fold CV accuracy
Raw	87,0%
Authors seed	86,2%
Llama chat seed42	81,4%
Llama chat seed43	91,8%
Llama chat seed44	96,8%

96,8% — CV на train. Опыт показывает способность конкретной Llama работать со знакомой плоской задачей.



Краткие итоги локальных LLM на small-v1

На малой выборке (5 000 товаров, 250 test-пользователей, 72 813 обучающих пары) Qwen7B, Llama8B и Qwen14B проверили по 20 наборов признаков через SQL/JSON-интерфейс. Test MAP@12 составил соответственно **1,10714%, 0,78034% и 0,74331%**, у Raw15 — **0,95963%**. Для получения 20 наборов потребовалось 67, 201 и 45 вызовов. Все результаты проверены повторным инференсом; устойчивое превосходство одной LLM над другой не подтверждено интервалами разностей.

У Qwen7B все 67 ответов читались как JSON; отказы были связаны с SQL и ограничениями интерфейса. В H003 проверили подробную диагностику, примеры и политику повторов. Последний вариант сократил число вызовов с 67 до 51, но test MAP снизился до **0,88874%**: улучшение исполнения не дало улучшения рекомендаций. Ошибочные программы отклонялись до обучения; выявленные ограничения диагностики описаны в исходном отчёте. H004 с полными SQL-запросами подготовлена, но подтверждённого результата нет.

В общем, с локальными моделями мне не повезло, поэтому в последующем я првоерю более умную модель gpt6.1-sol, а пока что перейду к ещё несколкьим рекомендательным алгоритмам.

7. Новая малая выборка small-v1: GNN, Recformer и TDM

Для ускорения исследований создан другой протокол: 5 000 покупателей / 5 000 товаров; итоговые train 4 804, validation 304, test 250. Деревья используют 72 813 одинаковых пар. Все методы оценивались по полному малому каталогу.

Да, выборка мала, но явное преимущество одного метода над другими могла дать картину того, какие методы совсем ущербны, а какие всё же стоят внимания.

7.1. Базовое сравнение на small

Метод	Validation, %	Test, %
Popularity	0,26256	0,72645
ID-GNN	4,41072	2,65341
ContextGNN	4,76193	3,01558
Two-tower GraphSAGE	1,55256	0,68368
LightGBM manual	0,99832	1,23388
LightGBM ID-GNN features	0,68531	0,96667

ContextGNN численно выше ID-GNN, но парный CI разницы включает ноль. Независимые инференсы и пересчёт MAP прошли. Проверялись целостность подвыборки, FK/time и CE value/gradient.

FK/time — проверки связей между таблицами и времени:

FK (foreign key, внешний ключ): покупки ссылаются на существующих покупателей и товары, ID после создания подвыборки согласованы.

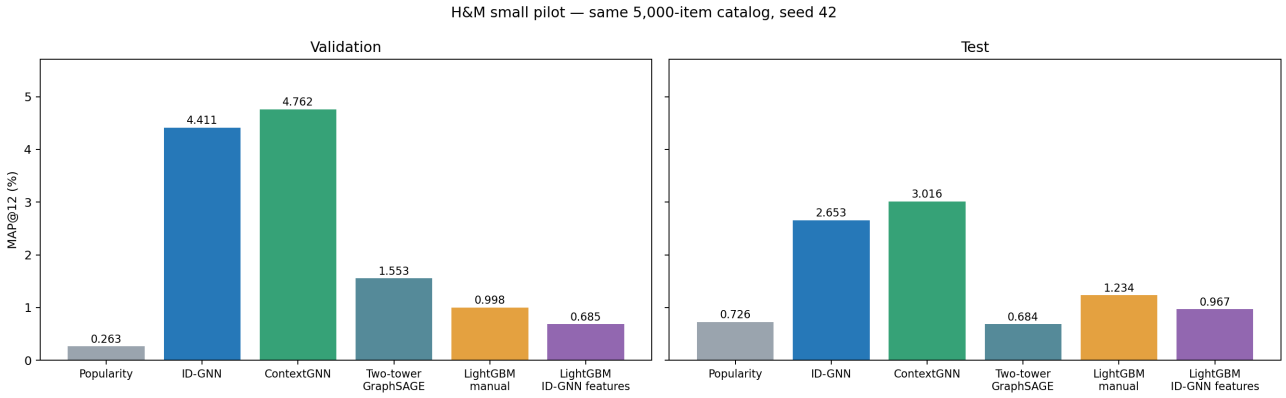
time: соблюдены временные границы. Например, неделя покупок, используемая как обучающая цель, заканчивается не позже начала validation.

CE value/gradient — проверка функции ошибки:

CE (cross-entropy, перекрёстная энтропия) — функция ошибки, по которой обучалась модель.

value — проверяли, что наша реализация возвращает такое же значение ошибки, как простая эталонная формула.

gradient — проверяли совпадение производных: модель получает такой же сигнал для обновления весов.



7.2. Recformer: initial fine-tuning, LR, freeze, дополнительные эпохи

Идея Recformer — представить каждый товар как текст из названий и значений его характеристик, а историю покупателя — как последовательность таких описаний. Transformer кодирует эту историю и формирует представление интересов покупателя, по которому ранжируются товары. Статья · Авторский репозиторий · Реплика.

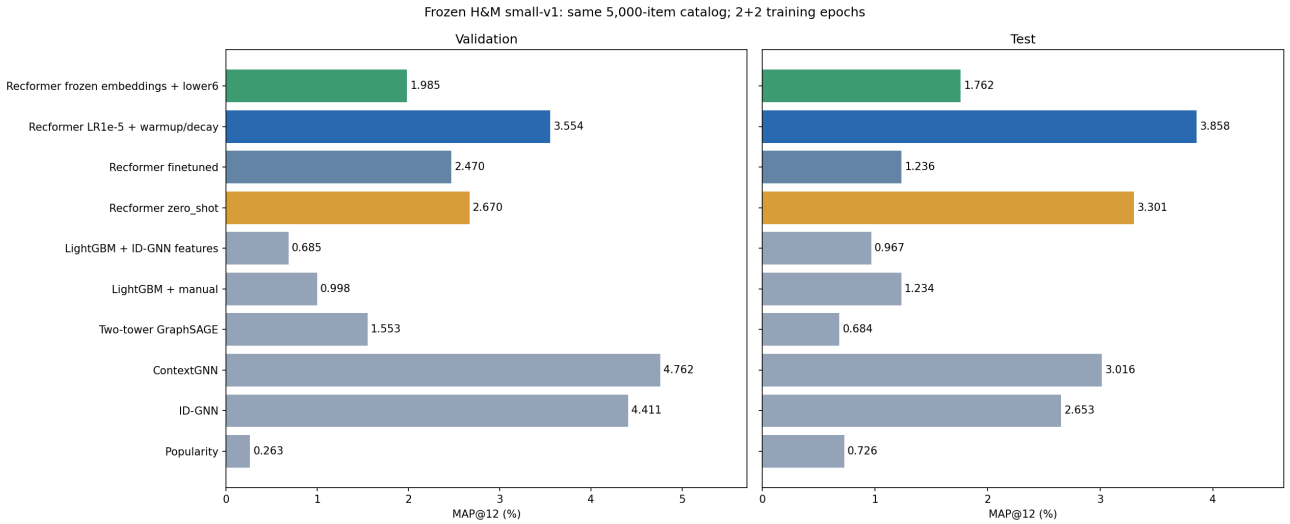
Использована репликация RecFormer, commit 26b00a58…, внешние предобученные веса и адаптация к недельным многопозитивным целям. Источник весов не аттестован как авторский checkpoint. Это адаптация, не воспроизведение статьи.

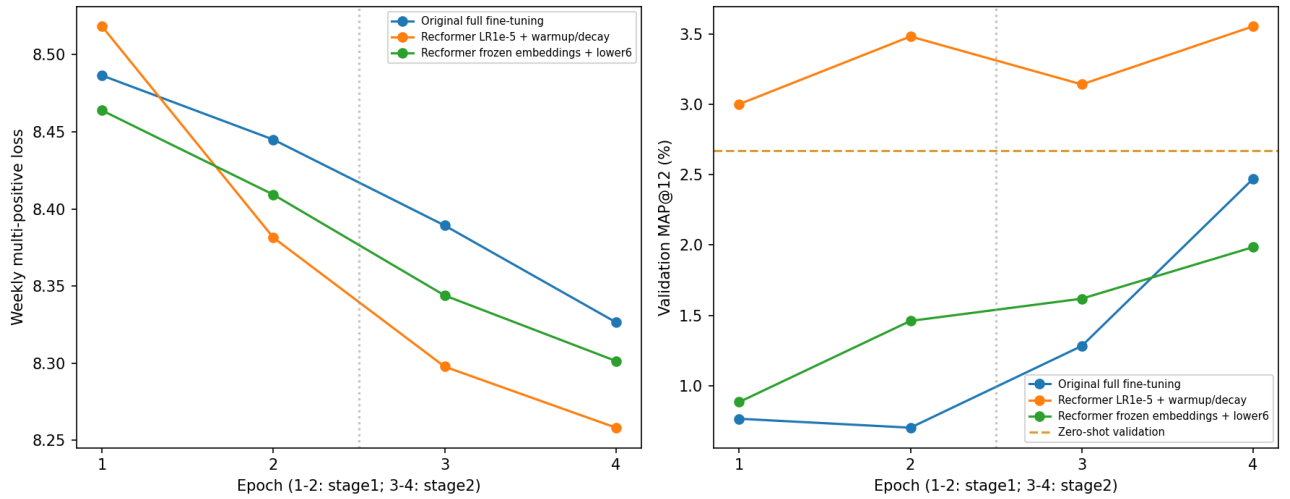
Вариант на small	Validation, %	Test, %	Результат
Zero-shot	2,66996	3,30054	Выше исходного fine-tuning
Исходное дообучение, 2+2 эпохи	2,47022	1,23556	Сильное ухудшение
LR 1e−5 + warmup100 + linear decay	3,55434	3,85751	Улучшение относительно исходного fine-tuning
Freeze embeddings + нижние 6 слоёв, constant LR 5e−5	1,98481	1,76156	По validation правило с epoch0 выбирает zero-shot
Короткий matched-контроль с LR floor	3,57807	3,43889	Те же веса, что у длинного
Длинный опыт с LR floor, остановка на 5 эпохах	3,57807	3,43889	Дополнительные эпохи не помогли

Уменьшение LR **вместе** с расписанием дало +2,62196 п.п. test к исходному дообучению, 95% CI [+0,59927; +4,81070]. Вклады LR и расписания отдельно не разделены. Преимущество 3,85751% над zero-shot или ContextGNN не подтверждено интервалами. Первоначальная причина деградации не установлена единственным причинным опытом.

Концентрация рекомендаций уменьшилась: исходный fine рекомендовал 600 различных товаров, LR/warmup — 1 757; самый частый товар встречался у 54,8% и 5,6% test-пользователей. Это наблюдение, не доказательство единственного механизма улучшения.

Длинный запуск планировал до восьми эпох, но остановился после пяти по patience3; короткий и длинный выборы взяли один checkpoint после второй эпохи. Поэтому увеличение реально проверенного горизонта **не улучшило выбранную модель**. Прежний low-LR run с другим decay не является matched-контролем числа эпох.





7.3. TDM: фиксированное дерево и кластеризация

TDM организует товары в дерево: листья — отдельные товары, внутренние узлы — их группы. Нейросеть оценивает интерес покупателя к узлам, а поиск идёт от крупных групп к конкретным товарам, отбрасывая менее перспективные ветви. Это позволяет искать рекомендации без оценки каждого товара каталога. Статья · Репозиторий Alibaba · Реализация TDM.

В эксперименте сравнивались:

Fixed — обучение с заранее заданным деревом товаров.

Cluster — перестроение дерева по сходству обученных векторов товаров, затем обучение новой модели с нуля.

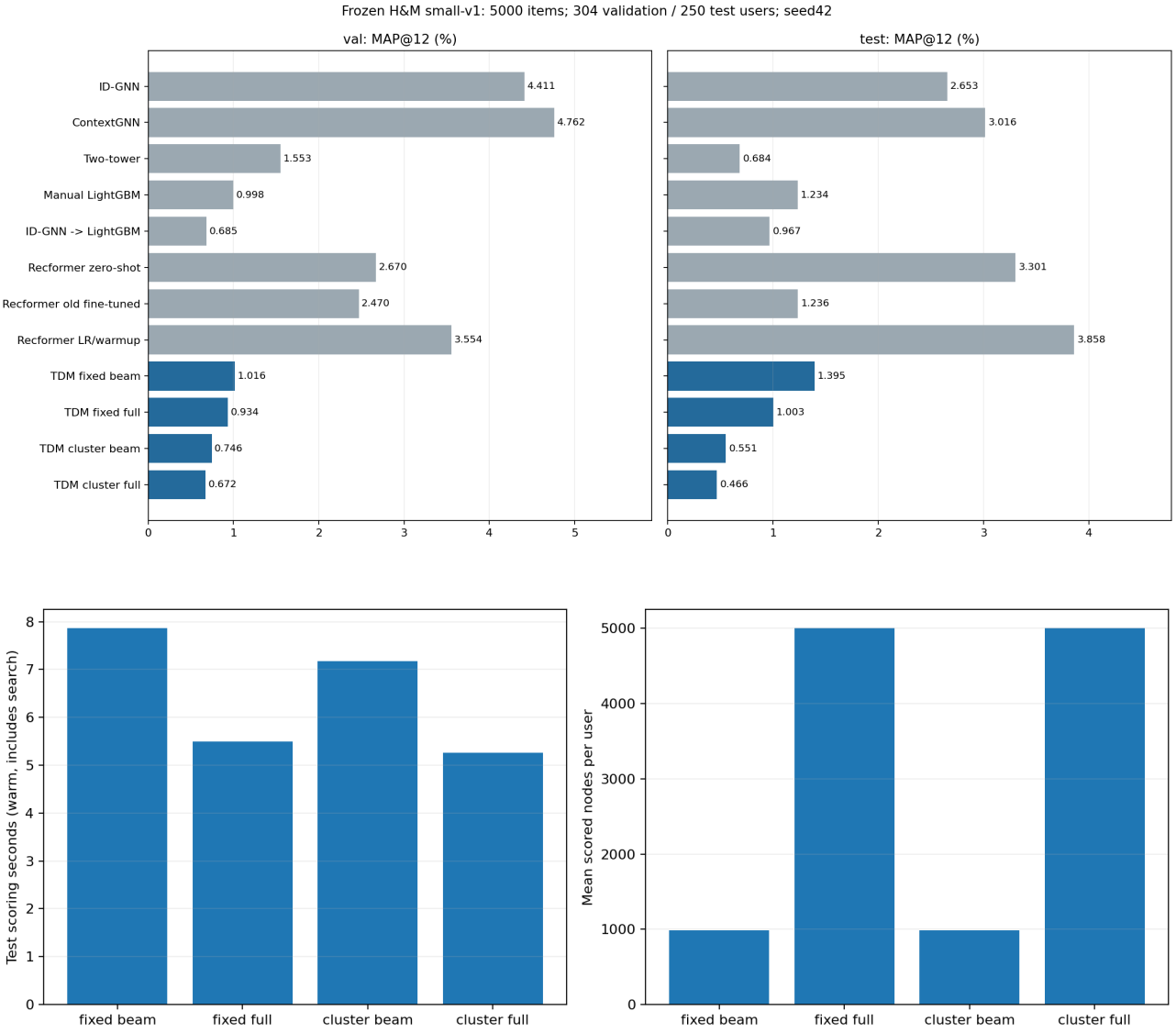
Beam64 — поиск сверху вниз с сохранением до 64 перспективных узлов на каждом уровне.

Все листья — оценка всех товаров теми же весами модели, чтобы проверить влияние отсечения ветвей при поиске.

Локальная PyTorch-адаптация, не неизменённый TorchEasyRec и не JTM. Fixed и cluster выбирались по beam64 validation до test. Cluster строился по train-обученным leaf embeddings fixed-модели, затем обучался с нуля.

Вариант	Validation, %	Test, %
Fixed, beam64	1,01601	1,39519

Вариант	Validation, %	Test, %
Fixed, все листья	0,93415	1,00292
Cluster, beam64	0,74588	0,55126
Cluster, все листья	0,67246	0,46639



8. Small-v1: API GPT и собственный поиск Codex

Downstream-условия сохранены: прежние train-пары и LightGBM, полный каталог 5 000, выбор по search/confirmation до финального test. Протоколы генерации отличаются, поэтому таблица описывает результаты процессов поиска.

Протокол	Входы дерева	Validation, %	Test, %	Объём поиска
H005 GPT, полный SQL	Generated + raw15	0,79144	0,97903	20 программ; 26 реальных POST
H006 GPT, Python	Generated + raw15	1,07971	1,35736	20 программ; 22 POST
H007, текущая сессия Codex	Generated + raw15	1,26060	2,23103	Ровно 20 наборов

Протокол	Входы дерева	Validation, %	Test, %	Объём поиска
H008 GPT, generated-only	Только возвращённые признаки	1,47856	1,42676	20 программ; 23 POST
H009 GPT, полная история своей популяции	Только возвращённые признаки	2,28699	0,72160	20 программ; 24 POST

H005: полный SQL и ложный отказ хоста

Исходный поток: 20 успехов, пять дубликатов, один ложный отказ, 26 POST. В исправленной оценке взяты первые 20 валидных программ из первых 24 ответов: четыре дубликата, ноль отказов; новых API-запросов не было. Победитель candidate004 и метрики не изменились.

H006: Python

22 POST: 20 программ, один отказ контракта, одна ошибка API. В v1 обнаружен другой порядок столбцов на пустой истории; до окончательного отбора v2 выровняла схему по именам и проверила empty/absent.

Test Python−SQL: **+0,37833 п.п.**, CI [−0,01463; +0,85118]; устойчивое преимущество не установлено. Все 12 признаков победителя независимо выражены SQL и дали точно те же float32 значения на всех 72 813 train-парах. Поэтому результат не доказывает принципиально большую выразительность Python.

20 AST-различных программ дали только **18 различных числовых матриц**.

H007: наборы текущей сессии Codex

Ровно 20 численно различных наборов, три популяционные линии. Победитель выбран по confirmation; test 2,23103%. В поиск входили частоты/давность, интересы категорий, цены, тренды и коллективные связи категорий по прошлым покупателям. Полная последовательность 20 гипотез записана в исходном отчёте.

Относительно провайдерского Python +0,87367 п.п., CI [+0,08423; +1,86397]. Но у текущей сессии были знания истории, инструменты и другой объём контекста/кода; её внутренний контекст не разделён на три независимые LLM. **Нельзя считать это честным испытанием одной и той же модели или доказательством подмены модели провайдером.**

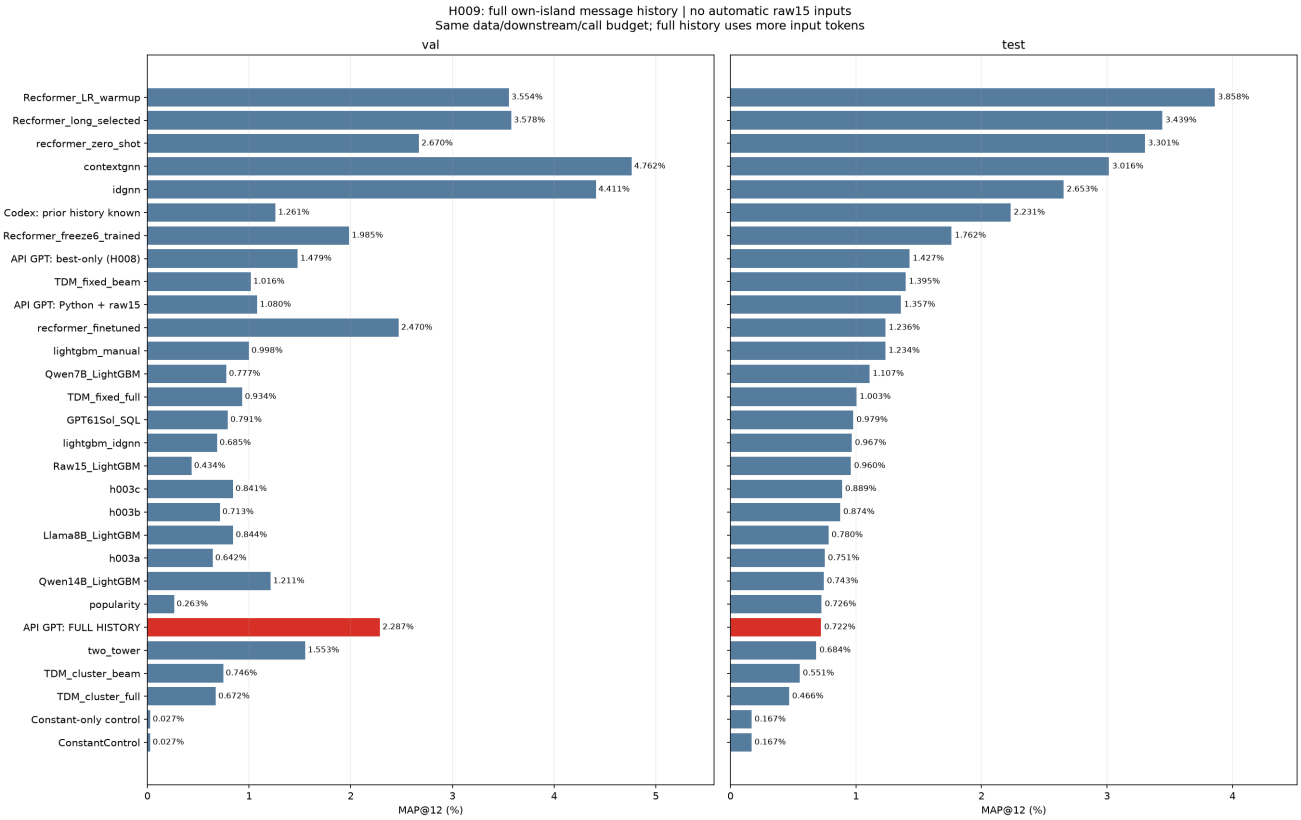
H008: только сгенерированные признаки

Host больше не добавлял raw15 автоматически; LLM могла явно вернуть исходные поля. У победителя 25 входов, семь — такие статические копии. 20 программ дали 19 численно различных матриц. Test−H006: +0,06940 п.п., CI [−1,03408; +1,27313]. Эффект удаления автоматического raw15 не подтверждён: это также новый стохастический поиск.

H009: полная история своей популяции

20 программ, 18 численных матриц. Validation выросла до 2,28699%, test упал до 0,72160%. H009−H008: −0,70516 п.п., CI [−1,36635; −0,15426]. Это отрицательный результат конкретного потока, не доказательство универсального вреда памяти.

Максимальный входной контекст 38 143 токена, 18 messages. Первые запросы H008/H009 побайтно одинаковы, но ответы уже различались до накопления истории: провайдерский seed не гарантировал общую траекторию. Один отказ вызван отсутствием hasattr у хоста, а не ошибкой Python модели; диагностическое разрешение показало численную эквивалентность следующему принятому ответу.



Какие-либо выводы по результатам на small делать рано, это были только пристрелочные запуски, поэтому теперь я расширил выборку и стал проверять уже с фиксированными условиями, которые позволяют полноценно сравнивать методы между собой.

9. Другая выборка medium: временное сравнение

Medium-v1 была подготовлена и отложена. **До обучения v2** протокол изменён: все исходные validation/test-цели и строки сохранены, недоступные товары не выбрасываются. V1 не является завершённым сравнительным экспериментом.

V2: другие 20 000 покупателей, каталог 10 000; три временных фолда; 5 519 test user-times. Модели и диалоги сбрасывались между фолдами. Earlier outer-недели могли войти в более позднюю историю: это заранее описанная rolling-origin оценка. В каждом API arm/fold — 20 принятых AST-программ; все шесть отборов фиксировались до outer-оценки.

Первоначально завершены 27 оценок девяти методов и replay. Позднее обнаружилось, что **исходные medium ID-GNN/ContextGNN-метрики невалидны**. Их результаты были невероятно малы, поэтому я заподозрил неладное и решил перепроверить их инференс.

Главная ошибка проекта: миллисекунды прочитаны как наносекунды

Validation/test имели datetime64[ms], а код делил integer timestamp на 10^9, как для ns. Cutoff становился около **1 597 секунд вместо 1,597×10^9**, графовые модели фактически лишались истории покупок. Простая проверка «нет будущих узлов» этого не обнаруживала: пустой временной контекст тоже проходил её.

Диагностика тех же весов fold0 после исправления только cutoff:

ContextGNN: test **0,0309%** → **1,4616%**.

ID-GNN: test **0%** → **1,0529%**.

Это прямое свидетельство влияния ошибки входов, но не итог refit: прежний выбор checkpoint тоже был испорчен. Поэтому отдельно обучены шесть новых GNN и выполнена новая оценка. Manual/raw/API SQL и история TDM/Recformer проверены отдельно: этой ms/ns-ошибкой они не затронуты.

Результаты medium после исправления и доступного follow-up

Метод на medium	Pooled test MAP@12, %
ContextGNN corrected, исходные 32/16	1,2791
ID-GNN corrected	1,1613
Recformer zero-shot	0,6286
Recformer tuned lr3e−6 / warmup300	0,5620
H008 / 20 наборов	0,4627
H009 / 40 наборов	0,4616
Recformer LR/warmup baseline	0,4525
H009 / 20 наборов	0,4514
H008 / 30 и / 40	0,4377
H009 / 30	0,4249
LightGBM manual	0,3473
TDM fixed beam64	0,2135
LightGBM raw15	0,1415

Pooled — среднее AP всех 5 519 строк, фолды имеют веса 1 839 / 1 982 / 1 698; это не простое среднее трёх MAP. Для исправленных GNN test по фолдам: ContextGNN 1,5325 / 1,1048 / 1,2081%; ID-GNN 1,4695 / 0,8939 / 1,1395%.

ContextGNN−ID-GNN: +0,1179 п.п., CI [−0,0016; +0,2397] — превосходство не подтверждено этой проверкой. H008/20−manual: +0,1154, CI [−0,0089; +0,2434] — также сигнал, не установленный выигрыш.

Продолжение API: 20 → 30 → 40 программ

Шесть поисков продолжены без обнуления бюджетов: всего 240 принятых программ, 18 выбранных budget×fold моделей. Все replays и независимые AP прошли. Это продолжения, а не независимые перезапуски.

Рост бюджета не дал монотонного test-прироста. Обнаружен **shortlist bottleneck**: на новом бюджете top3 search перестраивается и прежний confirmation-победитель может выпасть. На fold0 H008 call8 при бюджете20 имел confirmation 0,7180%; при30/40 исключён, победил call14 с 0,6966%. Сохранение incumbent предложено как новый протокол, но подтверждённого опыта с таким правилом нет. Исторические модели задним числом не перевыбирались.

H009/40−H009/20: +0,0102 п.п., CI [−0,0454; +0,0668]. H008/40−H008/20: −0,0250, CI [−0,0599; +0,0030]. Нельзя заключить ни гарантированную пользу, ни бесполезность большего бюджета вообще.

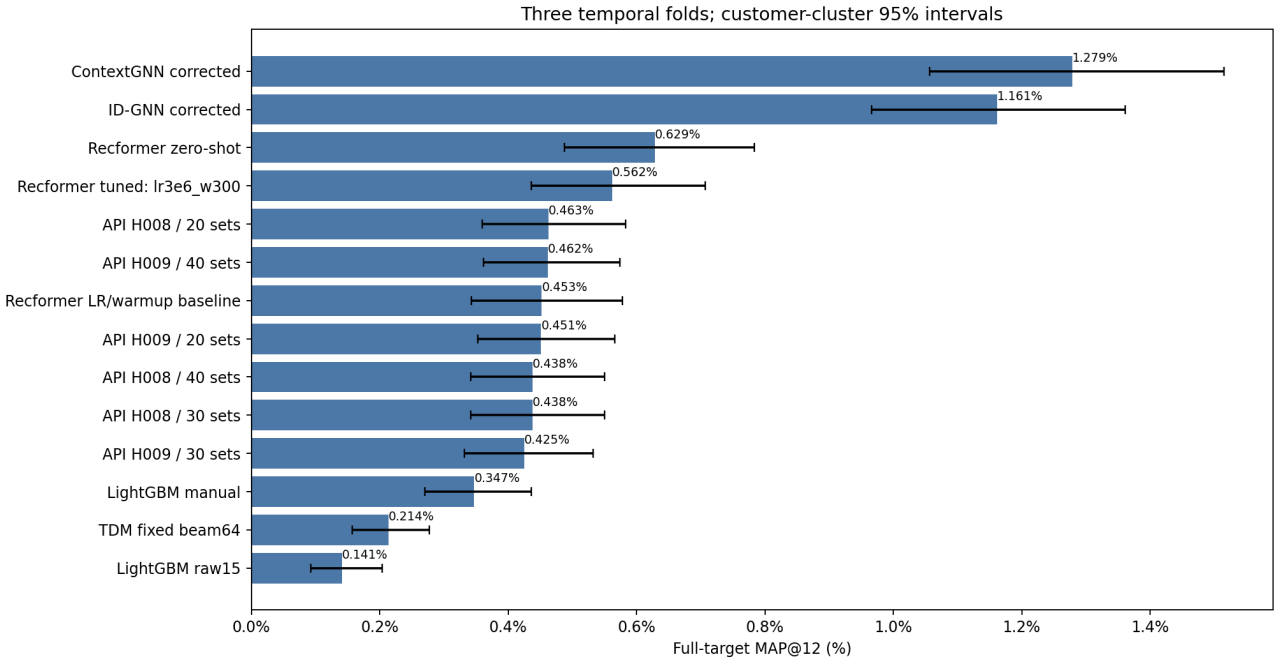
На исходном budget20 H009 потребила 602 739 известных input tokens против 139 937 у H008, около **4,31×**, без подтверждённого преимущества. AST-различных программ было по20; численных матриц H008 17/15/19, H009 20/17/18. Различие структур и значений учтено, отбор после аудита не менялся.

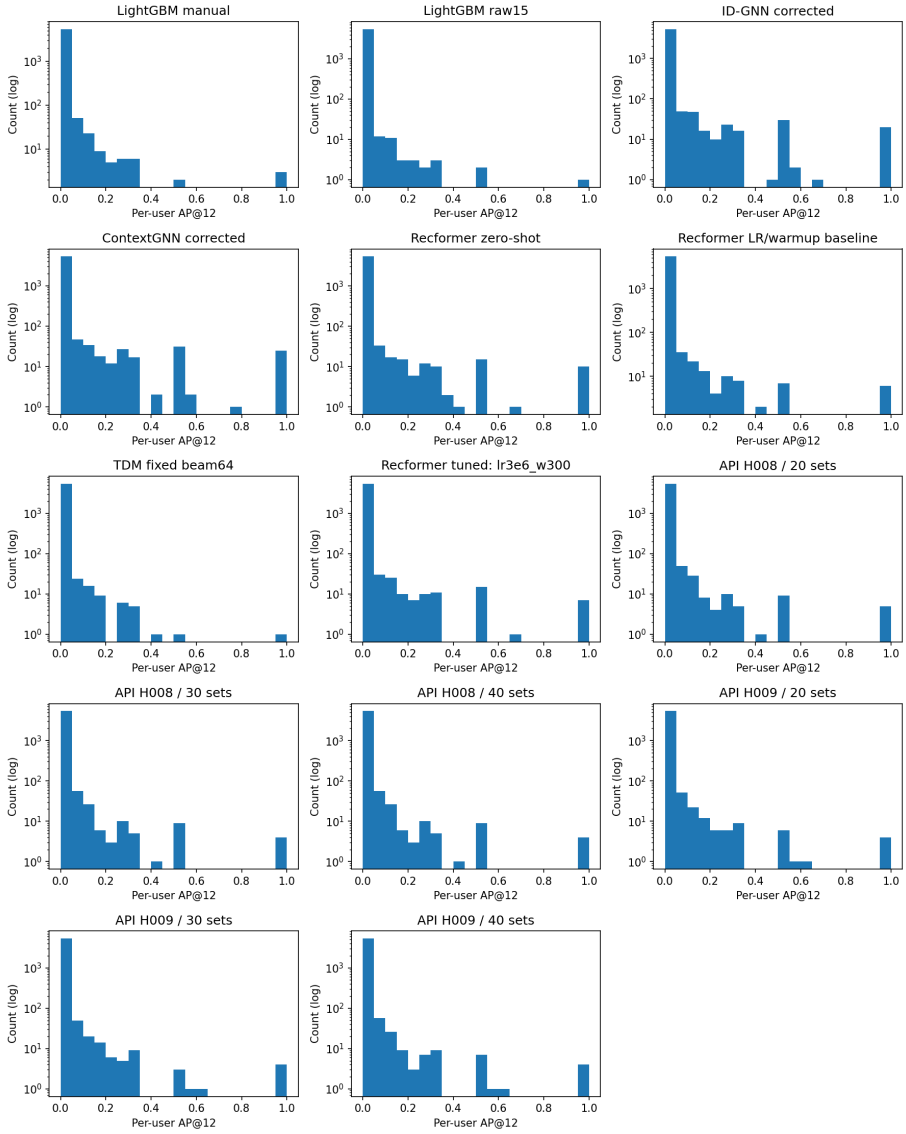
Recformer: незавершённая полная сетка, завершённая сводка доступного

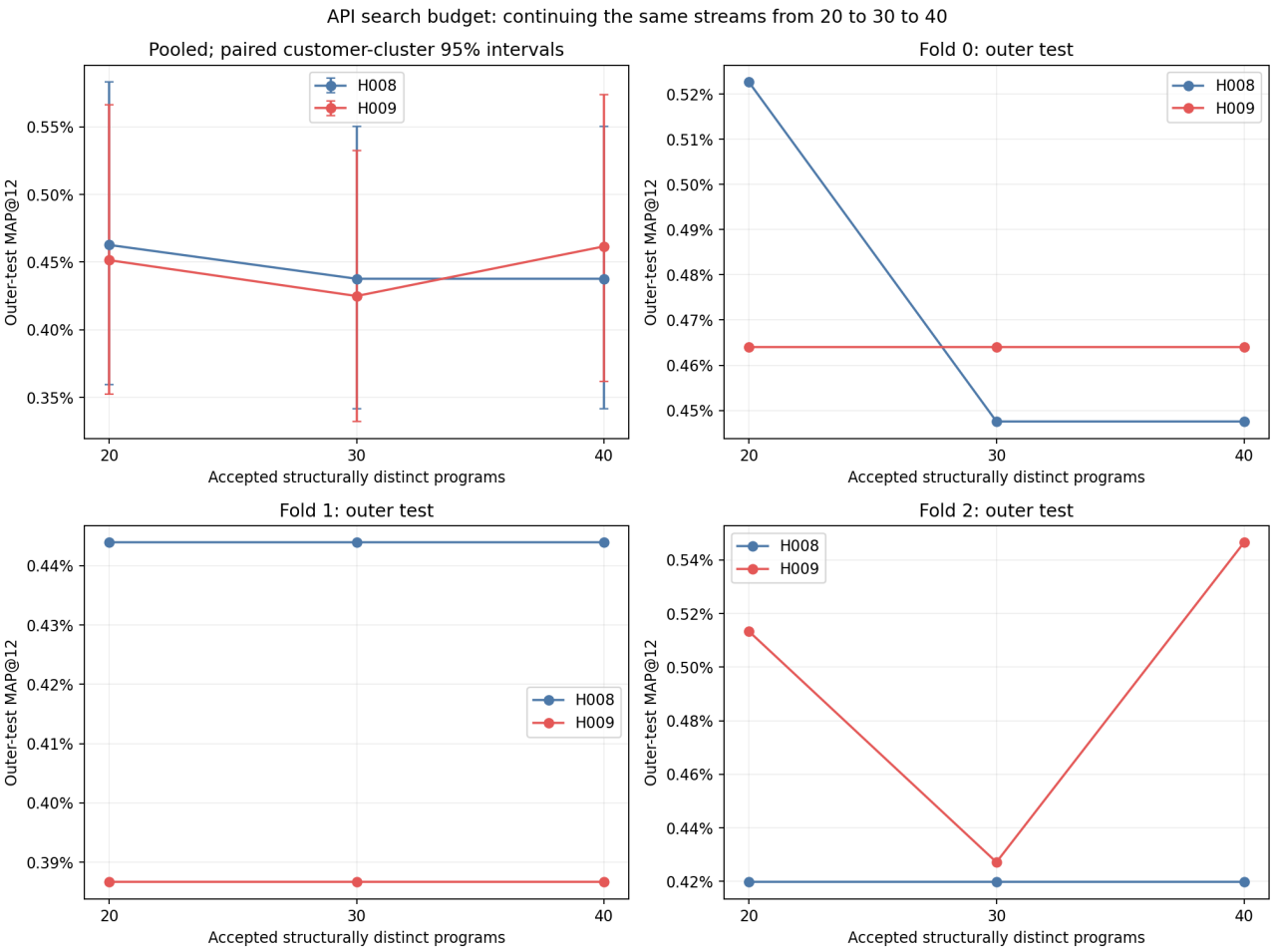
Планировались четыре новые LR/warmup-настройки на трёх фолдах. Получены три: lr1e−6/w100, lr3e−6/w300, lr1e−5/w300; **lr3e−6/w100 не запустилась в Kaggle**, ибо там какой то глюк, от которого я устал, поэтому решил не зацикливаться на этом эксперименте. Это отсутствие результата, а не плохое качество настройки.

Средний inner validation: baseline 0,5455%, lr1e−6/w100 0,6471%, lr3e−6/w300 **0,7151%**, lr1e−5/w300 0,6266%. По нему до новой test-оценки выбран lr3e−6/w300. Его test 0,5620% против baseline 0,4525%; разница +0,1095 п.п., CI [+0,0020; +0,2191]. Против zero-shot 0,6286% разница отрицательна, CI включает ноль. Полная исходная сетка **не завершена**, доступная сводка завершена.

Только три недели и один seed. Customer-bootstrap сохраняет все недели одного покупателя вместе; он не измеряет вариативность новых обучений и иных временных режимов. Последующие follow-up выполнялись на уже просмотренных outer-неделях.




LightGBM manual
LightGBM raw15
ID-GNN corrected
ContextGNN corrected
Recformer zero-shot
Recformer LR/warmup baseline
TDM fixed beam64
Recformer tuned: lr3e6_w300
API H008 / 20 sets
API H008 / 30 sets
API H008 / 40 sets
API H009 / 20 sets
API H009 / 30 sets
API H009 / 40 sets
Count (log)
Per-user AP@12



10. Medium: изменения ContextGNN и новые товары

ContextGNN сочетает два способа оценки товаров: для товаров внутри выбранного подграфа покупателя GNN вычисляет представления с учётом конкретной пары, а для остальных использует глобальную двухбашенную оценку по векторам покупателя и товара. Оценки согласуются в общем ранжировании. Товар вне подграфа не обязательно новый для покупателя: он мог просто не попасть в выбранную часть истории. Статья · Репозиторий.

Первые три проверки и их сочетание

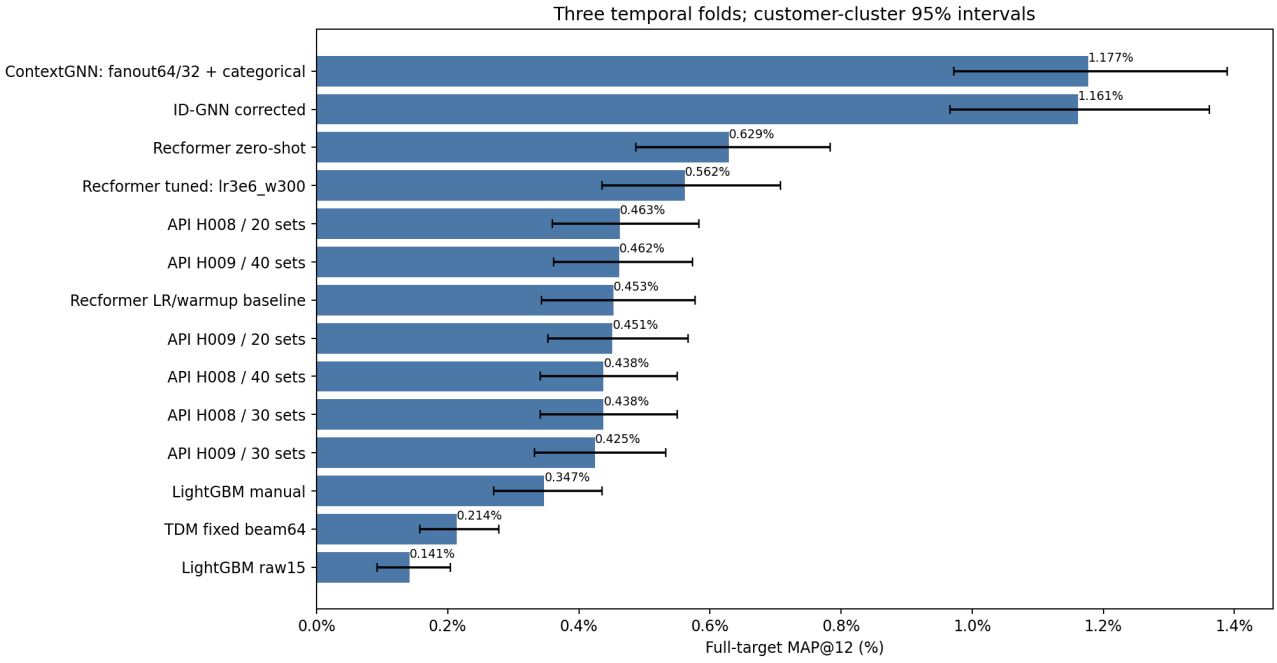
На прежних search-validation, всего 768 наблюдений, проверены: fanout64/32 вместо32/16, восемь item-кодов как categorical вместо numerical, обучаемая смесь local/global. Проведены девять новых fit и вмешательства в готовые веса.

Вариант	Pooled validation MAP, %
Исходный ContextGNN	1,1518
Fanout64/32, прежние веса	1,1772
Fanout64/32, refit	1,3852
Восемь categorical-кодов, refit	1,3329
Обучаемая смесь, refit	1,3843
Только global, прежние веса	0,0671
Фиксированная local-доля 0,25 / 0,50 / 0,75	0,3999 / 0,9971 / 1,1389

Вариант	Pooled validation MAP, %
Сочетание categorical + fanout64/32	1,4527

Сочетание−baseline на validation +0,3009 п.п., CI [+0,0126; +0,6520], но это неоднократно использованная выборка выбора эпох. На test сочетание дало **1,1766%**, ниже исходного corrected 1,2791%. Таким образом, validation-прирост не перенёсся в более высокий суммарный test MAP этого запуска. Отдельного test для каждого первого одиночного изменения в изученных итогах нет.

Широкий fanout при двух шагах главным образом возвращает более старые собственные покупки; он не добавляет путь к новым товарам через других покупателей. Это объясняет ограниченность данной локальной схемы, но не доказывает бесполезность более широких графовых моделей.



11. Medium: исследовательские инструменты H010 и автономный агент H011

H010: SQL-статистика train-данных

Опциональная v1 остановлена после пяти POST / четырёх полных предложений / двух принятых программ: LLM ни разу не использовала инструмент исследования. Outer-test не считался, программы не переиспользовались в v2. Это изменение протокола, не завершённый отрицательный test-результат.

В v2 каждая популяция обязана успешно запросить train-статистику до первой программы; вопросы выбирает сама GPT. Получены все 60 программ, 19 исследовательских SQL-запросов, 98 POST. Истории, изоляция, freeze, replay и независимый AP проверены.

Метод	Описание метода	Pooled test MAP, %
H009/20	GPT через API генерирует программы признаков для LightGBM, сохраняя полную историю сообщений своей популяции; отдельного инструмента исследования данных нет.	0,4514

Метод	Описание метода	Pooled test MAP, %
H010 research/20	Тот же подход дополнен SQL-инструментом: GPT запрашивает статистику обучающих данных и использует ответы при создании признаков.	0,4291
H011 Codex/20	Автономный Codex-агент исследует обучающие данные в изолированном терминале, пишет программы признаков и проверяет их через оценщик с LightGBM.	0,4307

Обозначение **/20** означает бюджет 20 принятых программ признаков на каждом временном разбиении, а не число эпох или API-запросов. Во всех трёх методах товары ранжирует LightGBM; LLM помогает искать признаки.

H010−H009: −0,0224 п.п., CI [−0,1327; +0,0880]. Улучшение не подтверждено. Фолды H010: 0,4207 / 0,3324 / 0,5509%. Известный usage 1 409 848 input / 82 813 output tokens, у пяти неуспешных попыток usage неизвестен.

H011: полноценная агентная среда

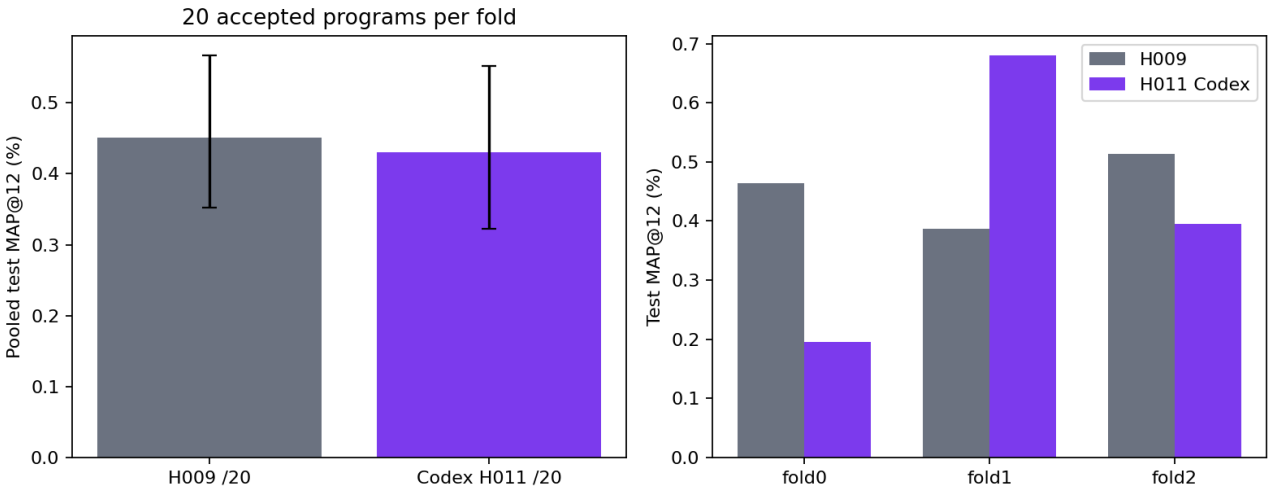
Подготовлен отдельный Codex CLI с train-only DuckDB и изолированным терминалом, отдельными профилями популяций, bounded API gateway, общим бюджетом оценщика, сохранением гипотез и программ. Smoke подтвердил доступ к train и отсутствие основного проекта в sandbox; это был инфраструктурный результат, не новая оценка качества.

После отдельного разрешения выполнен H011: **60 принятых программ**, 84 исследовательских terminal-вызова, 445 API POST. Все три выбора зафиксированы до новой outer-оценки, full replay/AP прошли. Фолды 0,1952 / 0,6795 / 0,3955%.

H011−H009: **−0,0207 п.п.**, CI [−0,1445; +0,1073]. Улучшение не подтверждено. Известны 10 931 402 input / 124 566 output tokens; у трёх попыток usage неизвестен.

Более дорогие и самостоятельные процессы в этой серии не дали подтверждённого прироста, но это не общий вывод о бесполезности агентов.

Аудит новых товаров для H010/H011 вместе с остальными методами приведён в разделе 13.



12. Medium: global loss, FEATURE, семантический граф и multi-interest

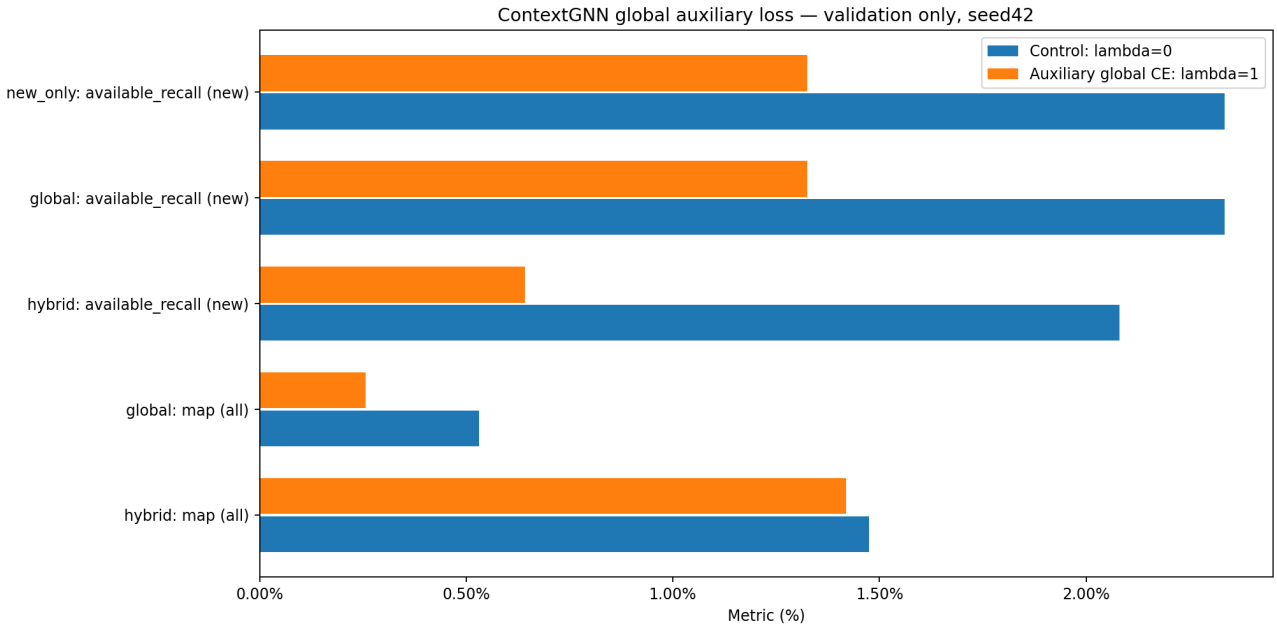
12.1. Дополнительный global CE, lambda=1

Основа — статья ContextGNN и код объединения локальных и глобальных оценок. Добавление отдельной ошибки глобальной ветки с коэффициентом lambda — **наша гипотеза из анализа этого кода**, а не готовое улучшение, воспроизведённое из статьи.

Шесть matched fit: контроль lambda0 и lambda1 на трёх фолдах, fanout64/32, categorical-коды. Эпоха выбиралась по hybrid MAP. Сравнение **только validation**, не test.

Метрика на validation	Контроль	+global loss
Hybrid MAP, %	1,4754	1,4194
Global-only MAP, %	0,5316	0,2565
Hybrid recall доступных новых, %	2,0810	0,6423
New-only recall доступных новых, %	2,3346	1,3252

Hybrid MAP-прирост не получен, CI включает ноль. Global MAP и оба приведённых recall ухудшились с отрицательными интервалами разности. Для данной lambda и реализации гипотеза не поддержана; другие коэффициенты не проверялись, test-результата этой серии нет.



12.2. FEATURE и семантический граф товаров

Режимы FEATURE/FUSION взяты из реализации ContextGNN: представление товара по признакам сравнивается с сочетанием признаков и ID-представления. Источники идеи графового обмена информацией между товарами — **FREEDOM** (статья, код) и **PinSage** (статья). FREEDOM использует фиксированный граф похожести по содержанию товаров; PinSage объединяет графовые связи и признаки. Наш TF-IDF/one-hot граф и способ его включения в ContextGNN — собственная упрощённая адаптация этих идей.

FEATURE удаляет lookup ID-компоненту RHS, локальный GNN остаётся. Семантический граф — фиксированные TF-IDF описаний + one-hot атрибутов, cosine top20, смесь .5 собственного FUSION-вектора + .5 соседнего. Изображений локально не найдено. Это адаптация идеи, не

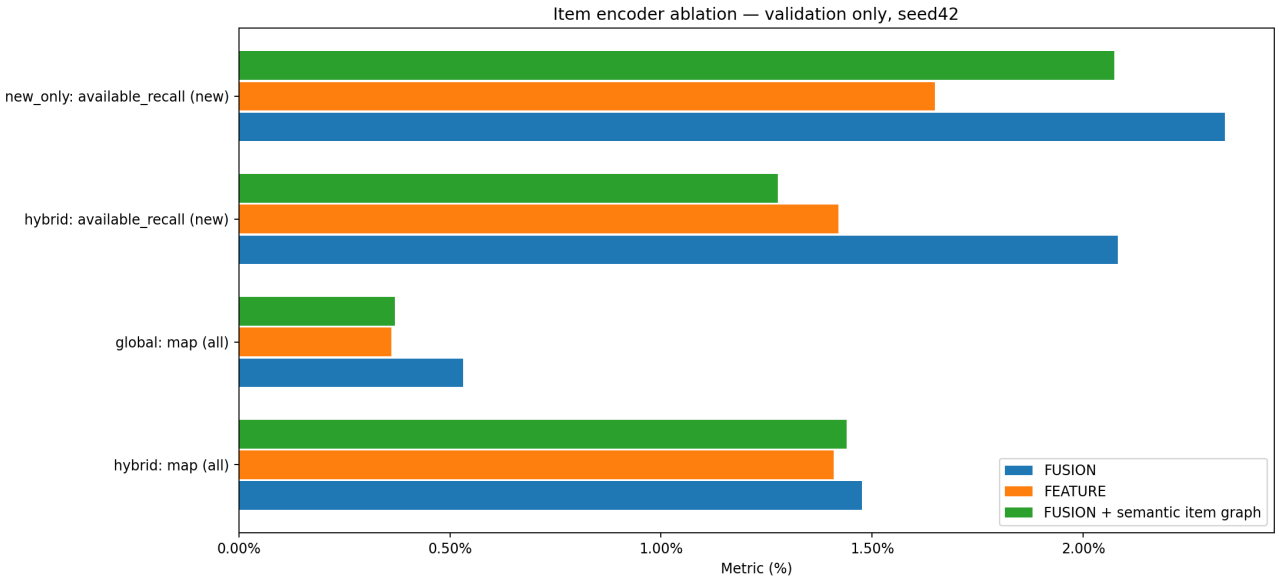
воспроизведение FREEDOM/PinSage.

Validation	Matched FUSION	FEATURE	FUSION + semantic graph
Hybrid MAP, %	1,4754	1,4084	1,4394
Global MAP, %	0,5316	0,3618	0,3692
Hybrid recall доступных новых, %	2,0810	1,4199	1,2762
New-only recall доступных новых, %	2,3346	1,6481	2,0726

На validation улучшение не получено. Для semantic graph global MAP и hybrid new recall имеют отрицательные CI разностей с matched FUSION. Остальные небольшие различия не подтверждены интервалами.

Позже отдельно выполнен test-инференс **FEATURE** из ранее выбранных checkpoint: pooled MAP **1,3111%**, CI [1,0913; 1,5411]%, фолды 1,3933 / 1,1091 / 1,4577%. Это численно выше исторического corrected 1,2791%, но такое сравнение меняет также fanout/типы признаков и не доказывает изолированную пользу FEATURE. Matched FUSION test 1,1595% появился в последующей общей сводке; сам порядок средних также не заменяет парный анализ и независимое подтверждение.

Аудит новых товаров для FEATURE вместе с остальными методами приведён в разделе 13. Подтверждённого test-результата semantic graph в изученной истории нет.



12.3. Один и четыре интереса

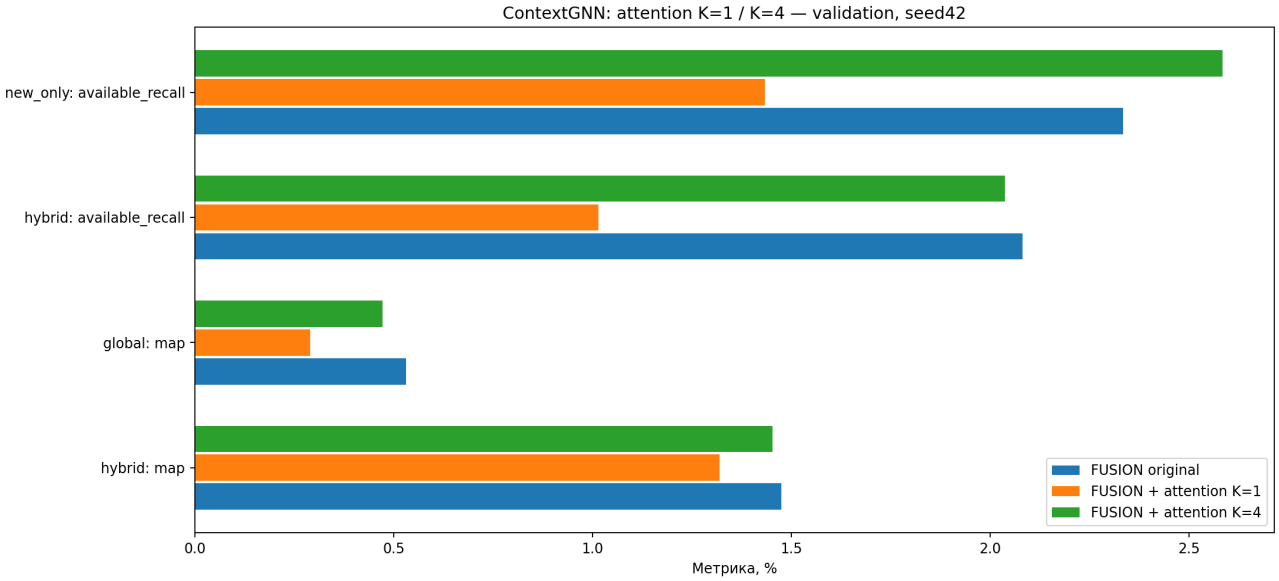
Источник идеи — **ComiRec**, который представляет интересы пользователя несколькими векторами: статья «Controllable Multi-Interest Framework for Recommendation», репозиторий, реализация моделей, включая ComiRec-SA. В проекте проверена адаптация attention-pooling этой модели к глобальной ветке ContextGNN, а не полное воспроизведение ComiRec.

Адаптация ComiRec-SA: attention по прошлым товарам sampled-графа, K векторов интереса, максимум dot product по интересам. От оригинального ComiRec отличаются последовательность, позиционные признаки, readout и отсутствие reranker разнообразия. Шесть новых fit K1/K4, три matched FUSION-контроля.

Вариант	Validation hybrid MAP, %	Test MAP, %	Test recall доступных новых normal / new-only, %
Matched FUSION	1,4754	1,1595	2,0417 / 2,4806
Attention K=1	1,3197	0,9677	1,2466 / 1,7343
Attention K=4	1,4528	0,9952	1,9878 / 2,3266

K4 лучше K1 по test new recall: +0,7412 п.п. normal, CI [+0,3801; +1,1236]; +0,5923 new-only, CI [+0,2037; +0,9958]. Но K4−matched FUSION по MAP **−0,1644 п.п.**, CI [−0,2974; −0,0348]; различия new recall с FUSION включают ноль. Global-only MAP практически одинаков: 0,2890% против0,2904%.

Четыре головы слабо специализировались: cosine запросов 0,99915–0,99981, attention 0,98846–0,99858; одна голова выигрывала 66–86% каталожных сравнений в зависимости от фолда. Четыре вектора не означают четыре независимых вкуса. Текущий вариант **не принят как улучшение FUSION**.



13. Единый аудит повторных и новых товаров для всех проверенных методов

Здесь объединены исходный novelty-аудит и поздние дополнения: **21 метод с сохранёнными результатами на medium test**. Это те же 5 519 наблюдений «покупатель — неделя», 4 548 покупателей и каталог 10 000 товаров. Модели и выбранные признаки зафиксированы; при аудите не было нового обучения или переотбора. Результаты full RelBench и small сюда не смешиваются.

Новый товар — товар, который конкретный покупатель ещё не покупал по полной доступной исходной истории до даты прогноза. Он не обязательно новый для магазина. **Доступный** — входящий в фиксированный каталог кандидатов. Всего в test 18 193 положительных user-item целей, доступны 6 733 (**37,01%**); новых целей 17 456, доступны 6 298 (**36,08%**). Недоступные цели сохранены в полных метриках.

Normal — обычный top-12, где разрешены повторные покупки. **New-only** — все ранее купленные товары исключены из кандидатов до нового выбора top-12; входная история модели при этом не удаляется. Recall показывает долю найденных новых целей, а не долю новых товаров среди рекомендаций. В таблице приведён условный macro Recall@12 доступных новых целей: среднее по **3 178 одинаковым наблюдениям** для всех методов. Полный recall новых, включая недоступные цели, считается по 5 358 наблюдениям; остальные 2 180 с исключительно недоступными новыми целями не входят только в условную метрику.

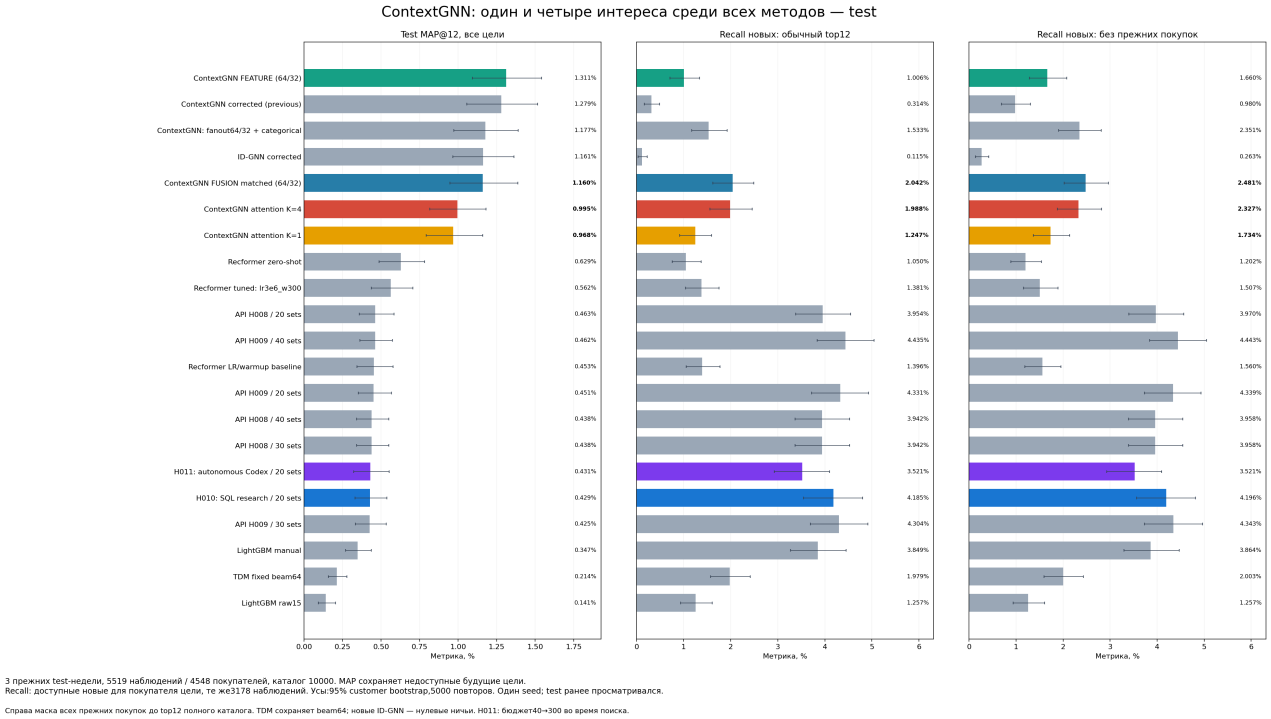
Метод	Test MAP@12, %	Доля MAP от повторных товаров, %	Recall доступных новых: normal, %	Recall доступных новых: new-only, %
ContextGNN corrected, исходный 32/16	1,2791	98,56	0,3136	0,9798
ID-GNN corrected	1,1613	99,54	0,1154	0,2633
Combined ContextGNN, categorical + 64/32	1,1766	89,45	1,5327	2,3511
ContextGNN FUSION, matched-контроль 64/32	1,1595	82,85	2,0417	2,4806
ContextGNN FEATURE	1,3111	92,54	1,0061	1,6605
ContextGNN attention K=1	0,9677	88,35	1,2466	1,7343
ContextGNN attention K=4	0,9952	82,54	1,9878	2,3266
Recformer zero-shot	0,6286	83,85	1,0497	1,2018
Recformer tuned, lr3e−6/w300	0,5620	72,40	1,3813	1,5072
Recformer LR/warmup baseline	0,4525	68,13	1,3960	1,5597
Manual LightGBM	0,3473	4,96	3,8485	3,8643
TDM fixed beam64	0,2135	21,35	1,9792	2,0028
LightGBM raw15	0,1415	10,63	1,2573	1,2573
H008/20	0,4627	2,20	3,9542	3,9699
H008/30	0,4377	2,12	3,9424	3,9581
H008/40	0,4377	2,12	3,9424	3,9581
H009/20	0,4514	3,68	4,3307	4,3385
H009/30	0,4249	2,69	4,3037	4,3430
H009/40	0,4616	3,58	4,4350	4,4429
H010 research/20	0,4291	3,01	4,1854	4,1958
H011 Codex/20	0,4307	7,46	3,5212	3,5212

MAP в таблице рассчитан по всем исходным целям в обычной выдаче. Его доля от повторных товаров — доля самой метрики, а не доля будущих покупок. Поэтому высокий общий MAP исходных GNN здесь преимущественно связан с повторными покупками, тогда как по recall новых целей выше API-признаки. Численный лидер recall — H009/40, но порядок средних сам по себе не доказывает устойчивого превосходства.

Исходный ContextGNN corrected, Combined и matched FUSION — разные сохранённые серии; их строки не объединены. Относительно первого у FEATURE изменены не только ID-компонента, но и fanout/типы признаков, поэтому это не чистая абляция. Для global CE и semantic graph есть validation-аудиты в разделе 12, но подтверждённых test-результатов нет: в эту таблицу они не добавлены. Методы, проверенные только на small или полном RelBench, также не имеют сопоставимого medium-аудита.

Для GNN, Recformer и деревьев new-only получен маской перед повторным top-12 по полному каталогу. У TDM сохраняется beam64: маска применяется к достигнутым листьям, недостигнутые цели остаются пропусками. У ID-GNN после исключения прошлых покупок содержательные локальные оценки исчезают; оставшиеся нулевые ничьи разрешаются по ID и не являются полноценной персонализацией новых товаров.

В парных проверках recall H010−H009/20 интервал включает ноль; для H011 обе разницы отрицательны. K4 улучшает recall относительно K1, но его различия с matched FUSION включают ноль.



3 прежних test-недели, 5519 наблюдений / 4548 покупателей, каталог 10000. MAP сохраняет недоступные будущие цели.
Recall: доступные новые для покупателя цели, те же3178 наблюдений. Усы:95% customer bootstrap,5000 повторов. Один seed; test ранее просматривался.

Справа маска всех прежних покупок до top12 полного каталога. TDM сохраняет beam64; новые ID-GNN — нулевые ничьи. H011: бюджет40→300 во время поиска.

Вывод

В наших экспериментах исходные графовые модели лучше справлялись с повторными покупками, а LightGBM с признаками, созданными API LLM, показывала более высокий recall новых для покупателя товаров. Графовые модели выигрывали по MAP за счёт повторных товаров, у них аж около 98,6% и 99,5% MAP приходилось на повторные товары, а у H009/40 — лишь 3,6%, то есть его качество преимущественно обеспечивали новые для покупателя товары. То есть несбалансированность полнейшая. Recformer показал промежуточный профиль: сохранял качество на повторных покупках и получал заметную часть MAP за счёт новых для покупателя товаров.

Интуиция мне подсказывала, что LLM должны дать больший профит, но результаты об этом не говорят.

Возможно, перспективно отдельно решать рекомендации повторных покупок и поиск новых товаров, дабы сбалансировать эффект проверенных методов.